Rendezvous Problem in Discrete Robotics

- Name: Archit Choudaha
- Institute: IISER Bhopal

In distributed robotics, cyclic pursuit and rendezvous are two main problems. These problems concentrate on the coordination of autonomous agents toward convergence at a point or in specific formations. Cyclic pursuit refers to a scenario where each robot follows its predecessor, thus implying global convergence if the visibility graph is connected. Rendezvous is an extension of this, where all the robots converge at the same point despite their different initial positions. It makes use of graph theory and Laplacian matrices to prove stability and convergence. These strategies show how local interactions can drive emergent global behavior, enabling scalable, robust solutions for multi-agent systems with limited sensing and communication.

The demonstration tells us the linear feedback strategies that stabilize robots to target formations by minimizing error functions. The derivations demonstrate how consensus can be achieved under conditions of limited sensor range and communication constraints. The derivation is as follows:

$$\dot{z} = vr$$
$$\dot{r} = s\omega$$
$$\dot{s} = -r\omega.$$

Let $\varepsilon > 0$. The point

$$p = z + \varepsilon r \tag{2.6}$$

is a distance ε in front of the unicycle, as shown in Fig. 2.11. Differentiate both sides of (2.6) with respect to t to get

$$\dot{p} = vr + \varepsilon s\omega.$$

Define u to be the right-hand side:

$$u := rv + \varepsilon s\omega. \tag{2.7}$$

Take inner products of both sides of this equation first with r and then with s. Since r, s are orthonormal, we obtain

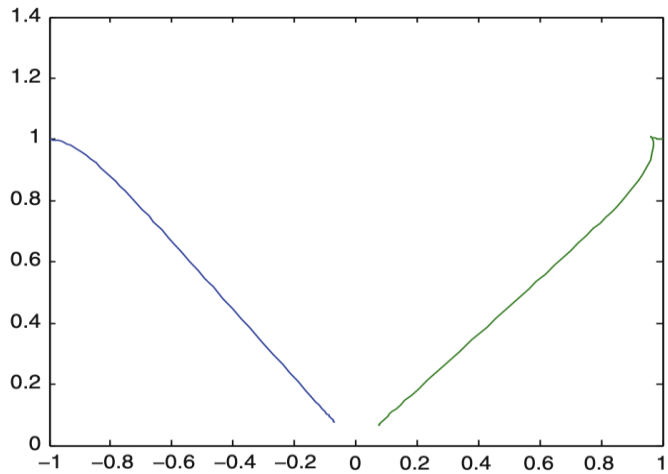
$$v = \langle u, r\rangle, \quad \omega = \varepsilon^{-1}\langle u, s\rangle. \tag{2.8}$$

The dynamics of the point p are simply

$$\dot{p} = u. \tag{2.9}$$

The cyclic pursuit result is followed by the MATLAB code to simulate the result for the varying ε:

Fig. 2.12 A unicycle controlled to go to the origin: $\varepsilon = 0.1$, $(x(0), y(0)) = (-1, 1)$ and $(x(0), y(0)) = (1, 1)$. The axes are x and y




```
clear; clc;
T=1000;
dt=0.1;
e=0.2;
x=1;
y=1;
theta=0;
x_traj=[];
y_traj=[];
figure;
hold on; grid on;
xlim([-1 1]);
ylim([0 1.4]);
title('Unicycle Simulation');

for i=1:T

    v=-x*cos(theta)-y*sin(theta)-e;
    w=(1/e)*(x*sin(theta)-y*cos(theta));
    x=x+v*cos(theta)*dt;
    y=y+v*sin(theta)*dt;
    theta=theta+w*dt;
    x_traj=[x_traj x];
    y_traj=[y_traj y];


    plot(x_traj,y_traj,'b')
    plot(-x_traj,y_traj,'g')
    pause(0.01)
end
```

An example to illustrate the graph:

Example 2.1 We illustrate by a numerical example. The task is to get a unicycle to drive to a beacon placed at the origin from any starting point. We start from the feedback linearized model (2.9). We can drive the point p to the origin by the control law $u = -p$, i.e., $u = -z - \varepsilon r$. From (2.8)

$$v = -\langle r, z\rangle - \varepsilon, \quad \omega = -\varepsilon^{-1}\langle s, z\rangle.$$

Now our control laws are

$$v = -x\cos(\theta) - y\sin(\theta) - \varepsilon, \quad \omega = \varepsilon^{-1}\left(x\sin(\theta) - y\cos(\theta)\right).$$

Figure 2.12 shows simulation results. Because the robot is not initially headed toward the origin, there are initial turns, followed by straight line segments. The robot does not meet the origin. To get it to end up closer to the origin, one would have to make ε smaller. Again, this makes ω have larger values, as can be seen from the formula

$$\omega(t) = \varepsilon^{-1}\left[x(t)\sin(\theta(t)) - y(t)\cos(\theta(t))\right].$$

For $t = 0,\ x(0) = 1,\ y(0) = 1,\ \theta(0) = 0$, we have $\omega(0) = \varepsilon^{-1}$. △

Cyclic Pursuit:

A collection of robots modelled as integrator points. The robots are numbered 1 to n. Robot 1 pursues robot 2; it in turn pursues robot 3; and so on; robot n − 1 pursues robot n; finally, robot n pursues robot 1. This strategy is called cyclic pursuit.

An example to understand cyclic pursuit is as follows:

Example 4.1 Consider four robots in cyclic pursuit. Model the robots as points in the complex plane: $z_1, \ldots, z_4 \in \mathbb{C}$. The neighbour sets are $\mathcal{N}_i(z) = \{i+1\}$ for $i = 1, 2, 3$ and $\mathcal{N}_4(z) = \{1\}$ and the differential equations are

$$\begin{aligned}
\dot{z}_1 &= z_2 - z_1 \\
\dot{z}_2 &= z_3 - z_2 \\
\dot{z}_3 &= z_4 - z_3 \\
\dot{z}_4 &= z_1 - z_4.
\end{aligned}$$

Define the vector and matrix

$$z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

Then the preceding four equations can be combined into

$$\dot{z} = (U - I)z. \tag{4.1}$$

The four trajectories can be found by familiar eigenvalue analysis. The characteristic polynomial of U is $s^4 - 1$, whose roots, i.e., eigenvalues of U, are $1, j, -1, -j$, the four roots of unity. Thus the eigenvalues of $U - I$ are the four points

$$0, -1 + j, -2, -1 - j$$

as shown in Fig. 4.2. For a given $z(0)$, the solution of (4.1) is the sum of four terms, one for each eigenvalue. The three terms for the left half-plane eigenvalues go to zero as t goes to ∞. The term for the zero eigenvalue is stationary. An eigenvector for

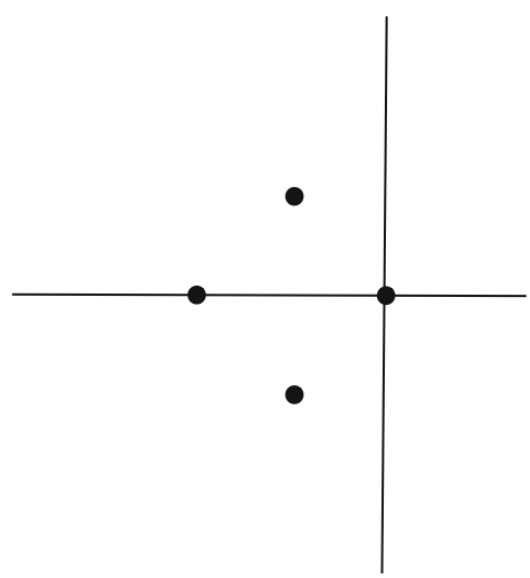


Fig. 4.2 The eigenvalues of $U - I$ are on the circle with centre -1 and radius 1

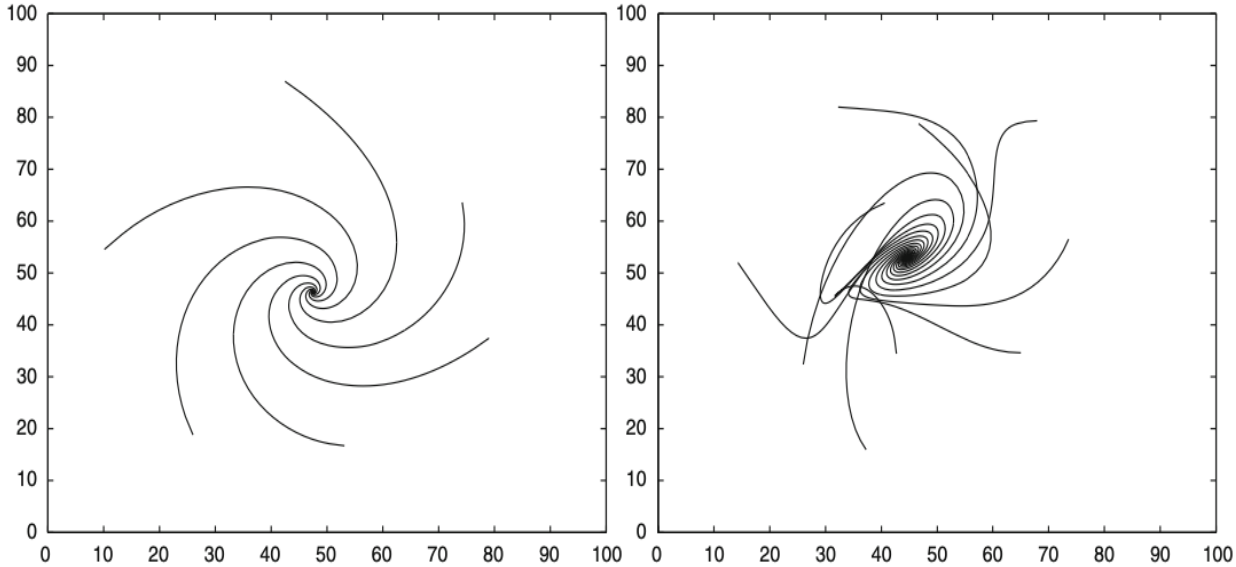


Fig. 4.3 Simulations of cyclic pursuit

this zero eigenvalue is $\mathbf{1}$, the vector of all 1's. Thus there exists a complex number c such that $z(t) \to c\mathbf{1}$, or $z_i(t) \to c$ for all i. This proves that the robots rendezvous.

Figure 4.3 shows two simulations (more than 4 robots) to illustrate the behaviour. We observe that the centroid is stationary, each robot converges to the centroid, and the trajectories sometimes intersect, but sometimes do not—it depends on the initial configuration. Furthermore, control is distributed in that the robots have identical local strategies and the sensor requirements are minimal: Only n information-flow links are needed for n robots. Also, there is an **emergent behaviour**—convergence to a common point. $\triangle$

More generally, consider finitely many, $n \geq 1$, robots with positions $z_1, \ldots, z_n$ in the complex plane. Let z denote the vector with components z_1 up to z_n. Let U denote the $n \times n$ real matrix with first row $0, 1, 0 \ldots, 0, 0$, second row $0, 0, 1, 0, \ldots, 0$, and so on, until the last row $1, 0, \ldots, 0$. Then the cyclic pursuit model is $\dot{z} = Mz$, $M = U - I$. In components,

$$\begin{aligned} \dot{z}_i &= z_{i+1} - z_i, \quad i = 1, \ldots, n-1 \\ \dot{z}_n &= z_1 - z_n. \end{aligned} \tag{4.2}$$

To simulate the results in figure 4.3, the MATLAB code is as follows:

```
clear; clc;
n=4;
z= [1;1j;-1;-1j];
dt= 0.01;
T = 20;
steps=T/dt;
epsilons = [0.05, 0.1, 0.5];
U = [0 1 0 0;
     0 0 1 0;
     0 0 0 1;
     1 0 0 0];
L=U-eye(n);
figure; hold on; grid on;
for eps = epsilons
    z_temp=z;
    trajectory =zeros(n, steps);
    for t = 1:steps
        z_temp= z_temp + dt*eps*L*z_temp;
        trajectory(:, t)= z_temp;
    end
    for i = 1:n
        plot(real(trajectory(i, :)), imag(trajectory(i, :)), 'LineWidth', 1);
    end
end
axis equal;
title('Cyclic Pursuit with Varying Epsilon');
xlabel('Real'); ylabel('Imaginary');
legend('\epsilon = 0.05','\epsilon = 0.1','\epsilon = 0.5');
```

Theorem 1:

System (4.2) has the property that the centroid of the robots' positions, $(1/n)\mathbf{1}^{\mathrm{T}}$, is stationary and for every initial condition, the robots asymptotically rendezvous at this centroid, i.e.

$$z_i(t) \to (1/n)1^T z(0) \;\; for\, all\, i \in 1, \dots, n.$$

Some Example of Cyclic Pursuit:

Example 4.2 Consider four robots in cyclic pursuit. The visibility graph is shown in Fig. 4.5. The neighbour sets are

$$\mathcal{N}_1 = \{2\}, \quad \mathcal{N}_2 = \{3\}, \quad \mathcal{N}_3 = \{4\}, \quad \mathcal{N}_4 = \{1\}.$$

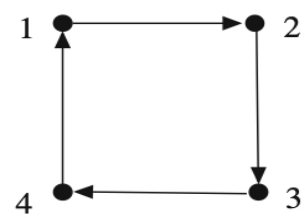


Fig. 4.5 Visibility graph for four robots in cyclic pursuit

The sensed outputs are

$$\begin{aligned} y_1 &= z_2 - z_1 \\ y_2 &= z_3 - z_2 \\ y_3 &= z_4 - z_3 \\ y_4 &= z_1 - z_4. \end{aligned}$$

Pick unity control gains, $F_i = 1$. Then the closed-loop system is given by

$$\dot{z} = FCz = \begin{bmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ 1 & 0 & 0 & -1 \end{bmatrix} z.$$

This is the cyclic pursuit strategy (4.2). The Laplacian matrix of the graph in Fig. 4.5 is

$$L = \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \\ -1 & 0 & 0 & 1 \end{bmatrix}.$$

Thus we have the identity $FC = -L$ and the dynamical system is therefore

$$\dot{z} = -Lz. \tag{4.4}$$

This is an important equation, showing that robot dynamics is related to the system's graph through the graph's Laplacian. △

Theorem 2:

Suppose the rendezvous problem is solvable for $\dot{z}_i = u_i$, $i = 1,2 \ldots . n$. *Then the above control law solves the formation stabilization problem. In other words, for every initial condition, the robots converge to a translated version of the polygon P.*

Let's see an example for the abovementioned theorem:

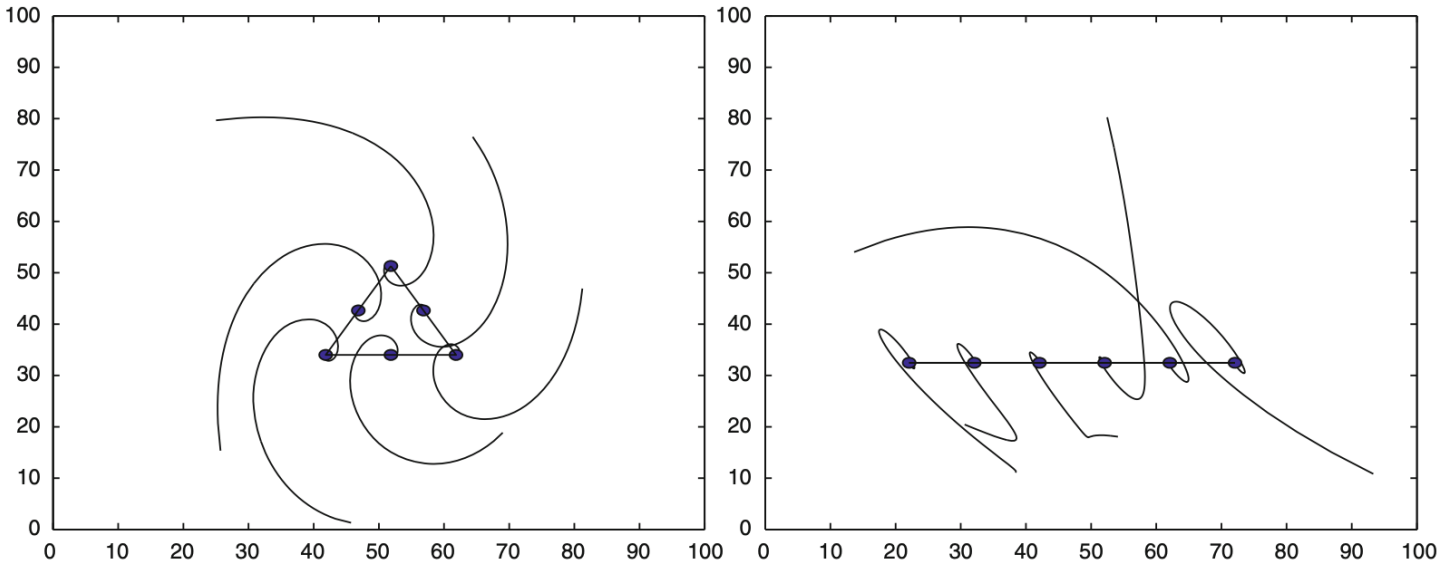
Example: Consider six robots in cyclic pursuit. We want to stabilize a triangular formation and a line formation. For the triangle, we choose:

$$c = \begin{bmatrix} 0 \\ 5 - j5\sqrt{3} \\ 10 - j10\sqrt{3} \\ -j10\sqrt{3} \\ -10 - j10\sqrt{3} \\ -5 - j5\sqrt{3} \end{bmatrix}, \text{ which gives } d = \begin{bmatrix} -5 + j5\sqrt{3} \\ -5 + j5\sqrt{3} \\ 10 \\ 10 \\ -5 - j5\sqrt{3} \\ -5 - j5\sqrt{3} \end{bmatrix}.$$

For the line:

$$c = \begin{bmatrix} 0 \\ -10 \\ -20 \\ -30 \\ -40 \\ -50 \end{bmatrix}, \text{ which gives } d = \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \\ -50 \end{bmatrix}.$$

Simulations for this example are mentioned below:



The MATLAB code to simulate the above results is as follows:

```
clc; clear; close all;
n=6;
triangle=[0; 5-5i*sqrt(3); 10-10i*sqrt(3); -10i*sqrt(3); -10-10i*sqrt(3); -5-5i*sqrt(3)];
line=[0; -10; -20; -30; -40; -50];
dt=0.01;
T=20;
steps=T/dt;
U=circshift(eye(n),1);
L=U-eye(n);
trajectory_triangle=zeros(n,steps);
trajectory_line=zeros(n,steps);
z_triangle=triangle;
z_line=line;

for t=1:steps
    z_triangle=z_triangle+dt*L*z_triangle;
    z_line=z_line+dt*L*z_line;
    trajectory_triangle(:,t)=z_triangle;
    trajectory_line(:,t)=z_line;
end

figure;
subplot(1,2,1);
hold on; grid on; axis equal;
for i=1:n
    plot(real(trajectory_triangle(i,:)),imag(trajectory_triangle(i,:)),'LineWidth',1.5);
end
title('Triangular Formation (6 Robots)');
xlabel('Real Axis'); ylabel('Imaginary Axis');

subplot(1,2,2);
hold on; grid on; axis equal;
for i=1:n
    plot(real(trajectory_line(i,:)),imag(trajectory_line(i,:)),'LineWidth',1.5);
end
title('Line Formation (6 Robots)');
xlabel('Real Axis'); ylabel('Imaginary Axis');
```

Bibliography

I would like to thank Dr. Arijit Sen, Asst. Prof. EECS Dept, IISERB, for guiding me through the course (LCS) and giving me an opportunity to learn about the Control System, Rendezvous problem and cyclic pursuit and giving me an opportunity to dive into the world of Discrete Robotics.

Below Mentioned are the sources used in the making of these projects:

1) The BODE Lecture by Dr. Bruce Francis on The Robot Rendezvous Problem.

2) IEEE Control Systems Society

3) Flocking and Rendezvous in Distributed Robotics book, by Dr. Bruce A Francis and Dr. Manfredi Maggiore

4) MATLAB